

31/08/21

Reading: Chapters 1,2,3 of 100 Pages Book

Introduction to ML

Rule-based classification: Uses expert's knowledge to determine thresholds for classification.

ML approach: Give data to ML algorithm to determine threshold for classification (supervised learning) or give data to ML algorithm to determine what to do with it (unsupervised learning).

Rules can break easily which is why the ML approach is preferred.

You give data to a learning algorithm to develop a model (in the case of threshold models, the threshold itself is the model). Then the input features for test data is given to the model which processes it via the inference algorithm to give you the output.

Learning algorithm gives you good values for the parameters to the model. For example, in 2D classification you have to fit a line through the data points so that the two classes are well separated. In this case the slope and intercept of the decision boundary line are the parameters for the model which will be given by the learning algorithm.

Supervised Learning: You have class labels and you are predicting test data to assign them a class label

Unsupervised Learning: You have no class labels and the learning algorithm identifies clusters in the data by grouping similar data points together. Once clustering is done, we can assign cluster IDs to new unseen data (test data).

Optimization Methods

Minimize the cost/error/loss function, etc. which will give better parameters for the model.

Unconstrained optimization or constrained optimization

Single minima (SVM) or multiple minima (Neural Nets)

02/09/21

Feature Extraction

Given an input, extract features from it. For example, given an image, how will you determine what are the useful features?

Features extracted must be relevant and can differ from problem to problem.

For spam or ham classification, histogram of spam keywords is an important feature.

Closed Form method

The method where we find maxima/minima via single and double derivative of the loss function/y

Gradient Descent

Iterative Method for Optimization.

We find new value of model parameters from old value as follows,

$$\theta(\text{new}) = \theta(\text{old}) + \Delta\theta = \theta(\text{old}) - \mu [J'(\theta)]_{\theta=\theta(\text{old})}$$

$$\Delta\theta = -\mu \cdot \left[\frac{\partial J(\theta)}{\partial \theta} \right]_{\theta=\theta(\text{old})}$$

θ = model parameters

$J(\theta)$ = Cost function which we have to minimize

μ = learning rate = the rate at which we try to move ahead in the space of the error function

Gradient Descent Numerical

Given the cost function $J(\theta)$ as below, minimize it using gradient descent.

$$J(\theta) = 1.2(\theta - 2)^2 + 3.2$$

$$J'(\theta) = 2.4(\theta - 2)$$

Take the initial value of $\theta = 1$. It doesn't matter too much what initial value you take but standard values like 1 may help you finish optimization faster.

Take two values of the learning rate, $\mu = 0.1$ and $\mu = 0.5$

Iteration 1:

$$\theta(\text{old}) = 1$$

Case 1: $\mu = 0.1$

$$\begin{aligned}\theta(\text{new}) &= \theta(\text{old}) - \mu [J'(\theta)]_{\theta=\theta(\text{old})} \\ &= 1 - 0.1 \cdot 2.4(1 - 2) \\ &= 1.24\end{aligned}$$

Case 2: $\mu = 0.5$

$$\theta(\text{new}) = 1 - 0.5 \cdot 2.4(1 - 2) = 2.2$$

You can check the new cost function values for these two θ values and compare it with the cost function value before iteration 1 as follows,

$$J(\theta = 1) = 1.2(1 - 2)^2 + 3.2 = 4.4$$

$$J(\theta = 1.24) = 1.2(1.24 - 2)^2 + 3.2 = 3.89$$

$$J(\theta = 2.2) = 1.2(2.2 - 2)^2 + 3.2 = 3.25$$

So you can see how doing even one iteration of gradient descent reduces the value of the cost function from the initial value of 4.4. However it drops even further in case 2 where we picked a bigger value of learning rate. However a bigger learning rate will end up giving us a minimum faster is not a sure thing.

Similarly, we can compute θ values for more iterations till we stop gradient descent.

When do we stop?

We set a threshold for the loss function, if it's sufficiently low then we stop. If model parameters don't change too much in an iteration (again a threshold), then too we can stop.

We can change learning rates after an iteration but there has to be a logic decided for it. Typically we start with comparatively larger learning rates and reduce it later on because initially we start with random values for the parameters and so taking larger steps on the cost function curve makes sense and later on, we'll reduce learning rate so that we take more precise steps on the curve so that we end up hitting the minima properly.

Note from the future: You can also have different learning rates for each feature. So you can define the collection of learning rates as a matrix.

Iterative method also computes the derivative, so why shouldn't we use the closed form method of optimization?

Because we don't have to solve the equation of $J'(\theta) = 0$, which is time consuming.

Cost function for 1D space

Model is given as, $y = ax + b$

We have to minimize sum of squared errors

Few points as to why we do square instead of something else like mod, etc.

Square gives us positive values, it is easily differentiable and gives more weight to even small errors as well.

Apart from that the squared error is equivalent to a Maximum Likelihood approach (more details later).

Mod gives us positive values but it is not differentiable which is a big no no.

$$J(a, b) = \sum_{i=1}^N [y_i - (a + bx_i)]^2$$

This is just computing (actual - predicted)² for each entry in the training dataset denoted by i and summing them all up to get cost function value.

Minimizing $J(a, b)$ in Closed Form

With one model parameter θ , we took its direct derivative. Here we have two parameters a and b , so we have to take partial derivatives of $J(a, b)$ wrt both a and b and set them to zero.

$$\frac{\partial J(a, b)}{\partial a} = 0$$

$$2 \cdot \sum_{i=1}^N [y_i - (a + bx_i)](-1) = 0$$

$$N \cdot a + b \sum_{i=1}^N x_i = \sum_{i=1}^N y_i$$

$$\frac{\partial J(a, b)}{\partial b} = 0$$

$$2 \cdot \sum_{i=1}^N [y_i - (a + bx_i)](-x_i) = 0$$

$$a \cdot \sum_{i=1}^N x_i + b \sum_{i=1}^N x_i^2 = \sum_{i=1}^N x_i y_i$$

Minimizing J(a, b) in Gradient Descent

In 1D model space (1 parameter only), the equation was,

$$\theta(\text{new}) = \theta(\text{old}) - u [J'(\theta)]_{\theta=\theta(\text{old})}$$

Now we have to adjust for two parameters a and b as,

$$\begin{bmatrix} a_{\text{new}} \\ b_{\text{new}} \end{bmatrix} = \begin{bmatrix} a_{\text{old}} \\ b_{\text{old}} \end{bmatrix} - \mu \begin{bmatrix} \frac{\partial J}{\partial a} \\ \frac{\partial J}{\partial b} \end{bmatrix}_{a=a_{\text{old}}, b=b_{\text{old}}}$$

2D Gradient Descent Numerical

Height and weight are given as below. Find the best fit line through this data using gradient descent. The line is given by, $h = mw + c$, where h = height and w = weight.

Height	Weight
163	60
170	65
181	83
176	73
178	76

Sol:

m and c are the model parameters here and w is the independent variable and h is the dependent variable.

Let m = 1 and c = 0 for starting value and let $\mu = 0.1$

$$\frac{\partial J(m, c)}{\partial c} = 2 \cdot \sum_{i=1}^N [y_i - (c + mx_i)](-1)$$

$$\frac{\partial J(m, c)}{\partial m} = 2 \cdot \sum_{i=1}^N [y_i - (c + mx_i)](-m)$$

Applying gradient descent.

Iteration 1:

$$\begin{bmatrix} m_{new} \\ c_{new} \end{bmatrix} = \begin{bmatrix} m_{old} \\ c_{old} \end{bmatrix} - \mu \begin{bmatrix} \frac{\partial J}{\partial m} \\ \frac{\partial J}{\partial c} \end{bmatrix}_{m=m_{old}, c=c_{old}}$$

Computing these is annoying, so fk it for now :)

07/09/21

We can extend whatever we wrote for 2D, i.e. one feature and one predicted to 3D, i.e. two features and one predicted.

So the line becomes a plane,

$$y = a + b \cdot x_1 + c \cdot x_2$$

And then you repeat the same things that you did in the Closed Form solution as well as the Gradient Descent solution, it's just that you will have to accommodate for the new parameter c.

How a typical ML Experiment works

Univariate Regression

- Training data
 - Features, Label $\{x_i, y_i\}; i = 1:N;$
 - Train Data features – $N \times 1$; Train Labels – $N \times 1$
- Test data
 - $\{x_{\text{test},m}, y_{\text{test},m}\}; m = 1:M;$
 - Test Data features – $M \times 1$; Test Labels – $M \times 1$
- Training: $\text{model} \leftarrow \text{learningAlgorithm}(\{x_i, y_i\})$
- Testing: $\{y_{\text{Pred}}_{\text{test},m}\} \leftarrow \text{inferenceAlgorithm}(\text{model}, \{x_{\text{test},m}\})$
- Evaluation:
$$\text{LSE} \leftarrow \text{evalPredictedLabels}(\{y_{\text{Pred}}_{\text{test},m}, y_{\text{test},m}\})$$

In the case of Multivariate Regression, say you have d features, each input x_i becomes a d -dimensional vector and X which represents the input features becomes an $N \times d$ matrix.

In this, all the magic happens in the learning algorithm which is the most complicated step.

Classification

In the case of Classification, what changes is the output label. Instead of an $N \times 1$ numerical output, we get an $N \times 1$ class output indicating which label it is.

- Training data
 - Features, Label $\{\mathbf{x}_i, C_i\}$; $i = 1:N$; $\mathbf{x}_i$ is d dimensional
 - Train Data features – $N \times d$; Train Labels – $N \times 1$
- Test data
 - $\{\mathbf{x}_{\text{test},m}, C_{\text{test},m}\}$; $m = 1:M$;
 - Test Data features - $M \times d$; Test Labels – $M \times 1$
- Training: $\text{model} \leftarrow \text{learningAlgorithm}(\{\mathbf{x}_i, C_i\})$
- Testing: $\{C_{\text{pred}_{\text{test},m}}\} \leftarrow \text{inferenceAlgorithm}(\text{model}, \{\mathbf{x}_{\text{test},m}\})$
- Evaluation:

$$\text{Accuracy} \leftarrow \text{evalPredictedLabels}(\{C_{\text{pred}_{\text{test},m}}, C_{\text{test},m}\})$$

Probabilistic Classifiers: Bayes Classifier

They estimate $P(C=C_k | \mathbf{x})$

Deterministic Classifiers give you a definite output as to which class **given input $\mathbf{x}$** belongs to whereas in Probabilistic Classifiers, it gives the probability of given **random variable $\mathbf{x}$** belonging to a given class C_k .

$$P(C = C_k | \mathbf{x}) = \frac{P(\mathbf{x}_1 | C = C_k) \cdot P(C = C_k)}{P(\mathbf{x}_1)}$$

$\mathbf{x}_1$ is continuous. Use Bayes Theorem assuming we have 1-D features

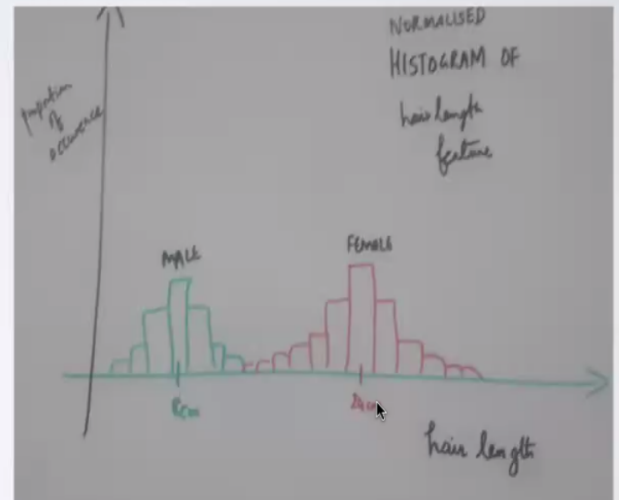
Fitting a Normalized Histogram

To fit the given continuous data, we can form histograms for each class label and determine the probabilities of each class's histogram for a new unseen test point, whichever class gives higher probability, we assign that point to belong to that class.

To make this more robust and have all the class label histograms scaled to the same extent, we normalize the histogram to have it centered around its mean.

$$P(x_1 | C = C_k)$$

To start with we can bin the continuous values and observe the histogram (after normalizing i.e. the elements sum up to 1)



It's a bit tiny but try to look at how the histogram is made for the two class labels "MALE" and "FEMALE" wrt the feature "Hair Length".

However, fitting the Histogram is a hassle because we will have to store all of the data and for each additional class label we will have to add one more histogram. So we will have to store the bin data for each window that we determine. For example, we bin given feature x into 20 discrete bins and we have 3 possible class labels, then we will have 3 histograms with 20 bins and for each bin we will have to store its frequency. So in total we will store $3 * 20 = 60$ data values just to determine where a new unseen data point can fall.

This gives us the motivation for fitting a Gaussian Distribution which can optimize this procedure.

Fitting a Gaussian Distribution

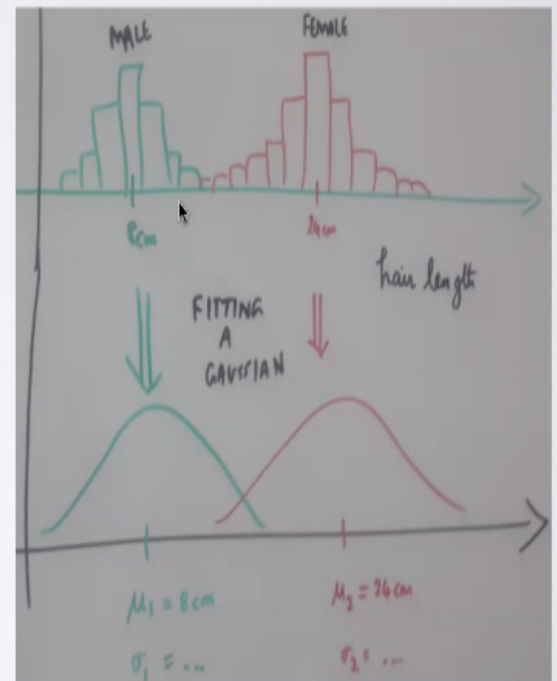
We fit a Gaussian Distribution over each class label and each Gaussian Distribution can be described with just two parameters, μ and σ and we don't have to store the original data, i.e. bins of the histogram.

So if there's 2 class labels, then we have to store $2 * 2 = 4$ parameters only!

Do note that this μ and σ refers to the sample mean and sample standard deviation and may or may not be representative of the population data which is why accuracy can get affected.

Makes sense to keep only two parameters μ and σ of the Gaussian distribution and throw away the original data

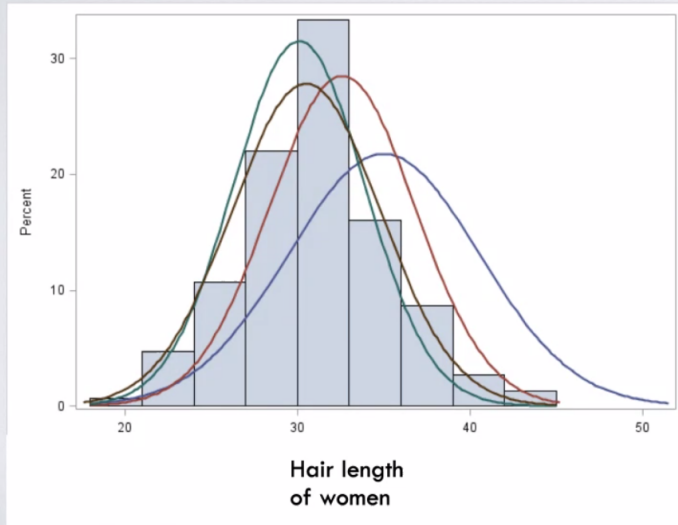
Question: How to estimate μ and σ ?



For all of this to work, we are assuming that the given data actually fits into a Gaussian Distribution or atleast can be approximated to a Gaussian.

So we are finding μ and σ for the Gaussian which MOST LIKELY fits this data (data = that class label's data).

DENSITY ESTIMATION TASK: WHICH GAUSSIAN IS THE BEST?



Data

$$\{x_i\}_{i=1:N}$$

Model

$$p(x_i | \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x_i - \mu)^2}{2\sigma^2}}$$

26

For this we need to find the best possible μ and σ that ensures that the given data point x_i from the given class label is most likely to fall on the Gaussian curve described by that μ and σ .

Maximum Likelihood Approach

$$p(X|\theta) = p(x_1 x_2 \dots x_N | \theta)$$

This is called the likelihood of X given θ , where, θ represents a single parameter, so in our Gaussian fitting case this θ will be replaced by μ and σ .

This probability measures how probable it is to get an input vector x_1 to x_N given the parameter value of θ (imagine θ to be the mean and $p(X|\theta)$ would make more intuitive sense, i.e. given a mean θ , what is the likelihood that this vector X has mean= θ).

We assume that all samples are independent, so we can compute $p(X|\theta)$ as,

$$p(X|\theta) = \prod_{i=1}^N p(x_i|\theta)$$

We would have gotten the best possible θ that gives us the best fit for given data X if we have maximized $p(X | \theta)$.

Maximizing $p(X | \theta)$ is fine but it becomes complicated for $p(X | \mu, \sigma)$, so to make the calculation easier, we take $\ln$ of the $p(X | \theta)$ to get $l(\theta)$ which is essentially the cost function we get.

$$l(\theta) = \ln(p(X|\theta)) = \sum_{i=1}^N \ln(p(x_i|\theta))$$

The product became summation because of the property, $\ln(a*b) = \ln(a) + \ln(b)$

Why did we take the ln of $p(X | \theta)$ and why is it valid without affecting the properties of the probability?

We are allowed to do it because the ln function is monotonic and maintains the maximas and minimas.

It is mainly for analytic simplification and not any physical implementation or intuition.

Finally, we have to maximize the value of $l(\theta)$ to get the best value of θ which we we'll represent as, $\hat{\theta}$

So our cost function is actually **maximizing the log-likelihood**

$$\hat{\theta} = \arg \max_{\theta}(l(\theta))$$

Maximum Likelihood Closed Form Solution

The only difference in the closed form solution is that we have to equate the differentiation of the cost function wrt θ to 0 which represented as,

$$\nabla_{\theta} l = \sum_{i=1}^N \nabla_{\theta} \ln(p(x_i|\theta)) = 0$$

And finding the solution to this would give us $\hat{\theta}$.

Derivation: Maximum Likelihood Closed Form Solution

$$p(x_i | \mu, \sigma) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

$$\ln p(x_i | \mu, \sigma) = \ln \frac{1}{\sigma \sqrt{2\pi}} + \ln e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

$$\ln p(x_i | \mu, \sigma) = -\ln(\sigma \sqrt{2\pi}) - \frac{(x_i - \mu)^2}{2\sigma^2}$$

$$\sum_{i=1}^N \ln(p(x_i | \theta)) = -N \ln(\sigma \sqrt{2\pi}) - \sum_{i=1}^N \frac{(x_i - \mu)^2}{2\sigma^2}$$

$$l(\theta) = \ln(p(X | \theta)) = \sum_{i=1}^N \ln(p(x_i | \theta))$$

$$\nabla_{\theta} l = \sum_{i=1}^N \nabla_{\theta} \ln(p(x_i | \theta)) = 0$$

$$l(\mu, \sigma) = \sum_{i=1}^N \ln p(x_i | \theta) = -N \ln(\sigma \sqrt{2\pi}) - \sum_{i=1}^N \frac{(x_i - \mu)^2}{2\sigma^2}$$

$$\frac{\partial l}{\partial \mu} = 0 \Rightarrow \sum_{i=1}^N (x_i - \hat{\mu}) = 0$$

$$\sum_{i=1}^N x_i = N \hat{\mu} \Rightarrow \hat{\mu} = \frac{\sum_{i=1}^N x_i}{N}$$

$$\hat{\sigma} = \sqrt{\frac{\sum_{i=1}^N (x_i - \hat{\mu})^2}{N}}$$

NOTE: The $\hat{\mu}$ and $\hat{\sigma}$ are the sample μ and σ which is what that hat represents. Understand this better.

HOMEWORK: UNDERSTAND HOW DO YOU DERIVE THOSE EQUATIONS